ESPM — Escola Superior de Propaganda e Marketing
Graduação em Sistemas de Informação — Data Integration

PROVA — Integração de Dados e Pipelines

Professor: André Insardi **Semestre:** 2026.1

Data: ___/___/2026 **Duração:** 2 horas

Aluno(a): _____ **Valor:** 100 pontos

INSTRUÇÕES

1. *Consulta livre (anotações, slides, livros, internet) e uso de IA (ChatGPT, Claude, Copilot etc.) são* ***permitidos****.*

2. *Respostas copiadas literalmente de IA ou de colegas, sem demonstração de compreensão própria, receberão* ***nota zero*** *na questão.*

3. *Todas as questões exigem* ***justificativa, raciocínio e/ou demonstração****. Respostas sem desenvolvimento não serão pontuadas.*

4. *Na questão de código, indique claramente* ***o que estava errado*** *e* ***por que*** *a correção resolve o problema.*

5. *Quando a questão pedir* ***código****, escreva o código completo e executável. Pseudo-código ou descrições textuais* ***não pontuam****.*

6. *Cada sub-item possui um* ***limite máximo de palavras****. Respostas que excedam o limite serão avaliadas apenas até o limite. Seja objetivo.*

7. *Gerencie seu tempo: a sugestão é ~25 min por questão.*

8. Figuras, códigos e links de fontes não contam no limite de palavras de cada questão.

OBS: Grifei os itens em amarelo para ficar mais fácil de achá-los em tamanha quantidade de texto.

Questão	Tema	Pontos	Nota
1	Fundamentos de ETL e Infraestrutura	20	
2	Ecossistema Big Data e Arquiteturas	25	
3	Depuração de DAG (Airflow)	20	
4	Pipeline ETL: Modelagem e Implementação	20	
5	Questão Integradora (Ensaio)	15	
Total		**100**	

Questão 1 — Fundamentos de ETL e Infraestrutura [20 pontos]

Uma fintech precisa integrar dados de 3 fontes distintas para alimentar seu data warehouse:

Fonte A: API REST de transações bancárias (JSON paginado, ~500 mil registros/dia, atualização contínua).

Fonte B: Banco PostgreSQL legado de cadastros de clientes (10 tabelas normalizadas, ~200 mil registros, atualização diária).

Fonte C: Planilhas Excel mensais de compliance enviadas por parceiros (formato variável, sem padronização).

O time de engenharia está debatendo entre usar ETL clássico (transformar antes de carregar) ou ELT moderno (carregar dados brutos e transformar no destino).

(a) [8 pts] Compare ETL vs ELT para ESTE cenário específico: [Máx. 200 palavras]

(i) Para cada uma das 3 fontes (A, B e C), indique qual abordagem (ETL ou ELT) é mais adequada e justifique com base no volume, formato e frequência dos dados.

(ii) Desenhe o fluxo de dados (pode ser textual, ex: API → Extract → Transform → PostgreSQL) mostrando ONDE ocorre a transformação em cada abordagem.

(iii) Em qual cenário o ELT seria PIOR que o ETL? Justifique com um exemplo concreto desta fintech.

(b) [7 pts] Idempotência e Upsert: [Máx. 200 palavras + SQL]

(i) Defina idempotência no contexto de pipelines de dados e explique por que ela é crítica para pipelines que rodam em produção.

(ii) Considere a tabela abaixo. Escreva o comando SQL completo (PostgreSQL) de INSERT com ON CONFLICT que implemente upsert, atualizando amount, status e updated_at quando a transação já existe.

(iii) Explique EXATAMENTE o que acontece na segunda execução do pipeline se a API retorna 1.000 transações, das quais 800 já existem no banco (com valores alterados) e 200 são novas.

Coluna	Tipo	Restrição
transaction_id	VARCHAR(36)	PRIMARY KEY
amount	NUMERIC(12,2)	NOT NULL
status	VARCHAR(20)	NOT NULL
updated_at	TIMESTAMP	DEFAULT NOW()

(c) [5 pts] Docker Compose e reprodutibilidade: [Máx. 150 palavras]

(i) No pipeline ETL da prática em aula, o docker-compose.yml define dois serviços (app Python + PostgreSQL). Explique o papel EXATO de depends_on com condition: service_healthy e o que acontece se essa configuração for removida.

(ii) Qual o risco de NÃO usar um volume para o PostgreSQL? O que acontece com os dados após docker compose down?

(iii) Por que variáveis de ambiente (.env) devem ficar FORA do código-fonte e FORA do controle de versão?

Questão 2 — Ecossistema Big Data e Arquiteturas de Dados [25 pontos]

Uma rede de supermercados com 200 lojas gera dados de 4 fontes distintas:

Fonte	Descrição	Volume	Tipo
PDVs	Transações de vendas das 200 lojas	~2 milhões/dia	Estruturado
Câmeras	Vídeos de segurança das lojas	50 TB/mês	Não estruturado
Redes sociais	Menções à marca no Twitter/Instagram	Tempo real (streaming)	Semi-estruturado
IoT	Sensores de temperatura em câmaras frias (a cada 30s)	~5,7 milhões/dia	Série temporal

(a) [10 pts] NoSQL — Para CADA uma das 4 fontes de dados acima: [Máx. 80 palavras por fonte, total 320 palavras]

(i) Indique qual família NoSQL é mais adequada (documento, chave-valor, colunar ou grafo). Cite um banco de dados específico dessa família.

(ii) Justifique a escolha com base no padrão de acesso e na estrutura dos dados.

(iii) Explique por que um banco relacional NÃO seria adequado para ESSE caso específico, citando ao menos uma limitação concreta (schema fixo, escala, latência, etc.).

(b) [8 pts] Arquitetura Lambda vs Kappa: [Máx. 230 palavras + diagrama]

(i) Desenhe (pode ser textual) a Arquitetura Lambda identificando as 3 camadas (Batch Layer, Speed Layer, Serving Layer) e o papel de cada uma.

(ii) Mostre como os dados de vendas (batch, fim do dia) e os dados de sensores IoT (stream, tempo real) fluiriam nessa arquitetura. Indique qual tecnologia concreta poderia ser usada em cada camada (ex: HDFS, Spark, Kafka, HBase).

(iii) A Arquitetura Kappa propõe eliminar a camada batch. Para ESTE cenário do supermercado, a eliminação seria viável? Justifique com trade-offs concretos de custo, complexidade e latência.

(c) [7 pts] Data Lake, Data Warehouse e Data Swamp: [Máx. 200 palavras]

(i) Diferencie Data Lake de Data Warehouse em EXATAMENTE 3 dimensões: momento do schema (on-read vs on-write), tipos de dados suportados, e perfil do usuário principal.

(ii) O que transforma um Data Lake em um Data Swamp? Cite 3 práticas concretas que causam essa degradação.

(iii) Proponha 3 medidas de governança de dados que o supermercado deveria adotar para evitar que seu Data Lake vire um Data Swamp. Para cada medida, explique COMO ela resolve um dos problemas do item (ii).

Questão 3 — Depuração de DAG: Apache Airflow [20 pontos]

Um colega implementou uma DAG no Airflow para executar um pipeline ETL que consome a API REST Countries (*restcountries.com/v3.1/all*) e carrega os dados no PostgreSQL. A DAG **roda sem erros de sintaxe**, mas apresenta problemas: as tasks executam fora de ordem, dados duplicados aparecem no banco a cada execução, e a DAG executa centenas de runs ao ser ativada. O código contém **5 erros** (conceituais e/ou de implementação). Identifique cada erro, explique o problema e apresente a correção.

```
1  # dags/etl_countries.py
2  from airflow import DAG
3  from airflow.operators.python import PythonOperator
4  from datetime import datetime
5  import requests
6  import psycopg2
7
8  dag = DAG(
9      dag_id="etl_countries",
10     start_date=datetime(2025, 1, 1),
11     schedule_interval="@daily",
12     catchup=True,                                  # <-- LINHA A
13 )
14
15 def extract():
16     response = requests.get("https://restcountries.com/v3.1/all")
17     data = response.json()
18     return data                                    # <-- LINHA B
19
20 def transform(data):                               # <-- LINHA C
21     cleaned = []
22     for country in data:
23         cleaned.append({
24             "name": country.get("name", {}).get("common"),
25             "region": country.get("region"),
26             "population": country.get("population"),
27         })
28     return cleaned
29
30 def load(records):                                 # <-- LINHA D
31     conn = psycopg2.connect(
32         host="etl_postgres", dbname="etl_db",
33         user="etl_user", password="etl_pass"
34     )
35     cur = conn.cursor()
36     for r in records:
37         cur.execute(
38             "INSERT INTO countries (name, region, population)"
39             " VALUES (%s, %s, %s)",
40             (r["name"], r["region"], r["population"])
41         )
42     conn.commit()
43     cur.close()
44     conn.close()
45
46 extract_task = PythonOperator(
47     task_id="extract", python_callable=extract, dag=dag)
48 transform_task = PythonOperator(
49     task_id="transform", python_callable=transform, dag=dag)
50 load_task = PythonOperator(
51     task_id="load", python_callable=load, dag=dag)
52                                                    # <-- LINHA E
```

(a) [14 pts] Identifique e corrija os 5 erros presentes no código (linhas A, B, C, D e E estão marcadas como dicas de localização — nem todas as linhas marcadas contêm erro, e pode haver

erros em linhas não marcadas). Para cada erro encontrado: [Máx. 50 palavras por erro, total 250 palavras + código]

(i) Indique a linha e transcreva o trecho com erro.

(ii) Explique conceitualmente por que está errado e qual o impacto no pipeline.

(iii) Apresente o código corrigido.

Dica: os problemas reportados são: (1) tasks executam fora de ordem, (2) dados duplicados no banco, (3) centenas de runs ao ativar a DAG. Pense em quais linhas causam cada sintoma.

(b) [6 pts] Reescreva a DAG usando a TaskFlow API (@task) do Airflow 2.x: [Máx. 150 palavras + código]

(i) Mostre o código completo da DAG refatorada com decorators @dag e @task.

(ii) Explique como XComs funcionam IMPLICITAMENTE na TaskFlow API (retorno de função → XCom push, parâmetro → XCom pull) vs EXPLICITAMENTE com PythonOperator (ti.xcom_push / ti.xcom_pull).

(iii) Qual a limitação de XComs para grandes volumes de dados (ex: 250 países com 50 campos cada)? O que fazer quando o volume excede o recomendável?

Questão 4 — Pipeline ETL: Modelagem e Implementação [20 pontos]

Considere o pipeline ETL da atividade do Banco Mundial. A API pública (api.worldbank.org/v2) fornece indicadores socioeconômicos para ~220 países. O destino é um banco PostgreSQL com 3 tabelas (countries, indicators, wdi_facts). O JSON de um indicador tem a seguinte estrutura:

```
{
  "indicator": {"id": "NY.GDP.PCAP.KD", "value": "GDP per capita..."},
  "country": {"id": "BR", "value": "Brazil"},
  "countryiso3code": "BRA",
  "date": "2022",
  "value": 8897.45,    // pode ser null
  "unit": "",
  "obs_status": "",
  "decimal": 0
}
```

(a) [8 pts] Modelagem relacional — Escreva o DDL completo (CREATE TABLE) das 3 tabelas: [Máx. 100 palavras + SQL]

(i) Tabela countries: defina as colunas (iso2_code, iso3_code, name, region, income_group, capital, longitude, latitude, loaded_at), com tipos corretos, PK e restrições.

(ii) Tabela wdi_facts: defina as colunas (iso2_code, indicator_code, year, value, loaded_at) com a chave primária COMPOSTA correta e as FOREIGN KEYS.

(iii) Explique por que a PK composta (iso2_code, indicator_code, year) é necessária: o que aconteceria se a PK fosse apenas iso2_code?

(b) [6 pts] Transformação — O JSON bruto da API contém problemas. Para CADA regra abaixo, escreva a função Python correspondente: [Máx. 150 palavras + código]

(i) T1 — Filtro de entidade: a API retorna países reais E agregados regionais (ex: 'EAS' = East Asia, 'WLD' = World). Países reais têm ISO2 com exatamente 2 caracteres. Escreva uma função is_real_country(record) que retorne True apenas para países reais.

(ii) T2 — Conversão segura: o campo value pode ser null, string, int ou float. Escreva safe_float(value) que retorne float ou None sem lançar exceção.

(iii) T3 — Filtro temporal: manter apenas registros com year entre 2010 e 2025. Escreva a lógica de filtro.

(c) [6 pts] Carga e validação: [Máx. 150 palavras + SQL/código]

(i) Escreva o código Python com SQLAlchemy que implementa o upsert na tabela wdi_facts usando pg_insert e on_conflict_do_update. Mostre quais colunas são atualizadas em caso de conflito.

(ii) Explique por que a carga de countries e indicators deve ocorrer ANTES da carga de wdi_facts. O que acontece se essa ordem for invertida?

(iii) Escreva 2 queries SQL de validação: uma que conte o volume por indicador e outra que verifique a idempotência (COUNT deve ser idêntico após segunda execução).

Questão 5 — Questão Integradora (Ensaio) [15 pontos]

Você foi contratado(a) como engenheiro(a) de dados por uma operadora de saúde que precisa construir um pipeline de integração de dados para alimentar um painel de indicadores clínicos. As fontes de dados são:

• **Sistema HIS (Hospital Information System):** banco Oracle com 500+ tabelas, dados de internações, procedimentos e alta.

• **API de laboratórios parceiros:** REST/JSON, resultados de exames com paginacão, ~50 mil registros/dia.

• **Planilhas da ANS:** indicadores regulatórios trimestrais em Excel, formato variável entre períodos.

Redija um **parecer técnico** que aborde **obrigatoriamente** todos os seguintes pontos. [Máximo 500 palavras no total — seja direto e técnico]

1. Arquitetura de dados: Proponha a arquitetura de integração (Data Warehouse, Data Lake ou Lakehouse). Justifique a escolha. Defina as camadas (raw/staging/trusted ou bronze/silver/gold). Indique onde cada fonte seria ingerida e em qual formato.

2. Pipeline ETL/ELT: Para cada fonte, descreva a estratégia de extração (full load vs incremental vs CDC), transformação (regras de qualidade, padronização, deduplicacão) e carga (upsert, idempotência). Justifique a escolha ETL vs ELT para cada fonte.

3. Orquestração: Proponha como orquestrar os pipelines (Airflow, cron, ou outro). Desenhe a DAG principal (pode ser textual). Defina o schedule, retry policy e alertas. Explique por que cron sozinho seria insuficiente para este cenário.

4. Governança e LGPD: Dados de saúde são dados pessoais sensíveis (Art. 5º, II da LGPD). Discuta: quais dados precisam ser anonimizados/pseudonimizados, estratégia de retenção, controle de acesso ao Data Lake, e auditabilidade do pipeline.

> *Critérios de avaliação: profundidade técnica (5 pts), integração entre os tópicos (4 pts), coerência e redação (3 pts), originalidade da análise (3 pts). Respostas genéricas ou copiadas de IA sem personalização receberão até 50% da pontuação.*

***Boa prova!** Lembre-se: o diferencial não é ter acesso à informação, mas saber compreendê-la, aplicá-la e criticá-la.*